

Aula 7 — Interrupções e ruído de contato

O mecanismo que a Q4 da integradora pediu

Microcontroladores — DENE/UFMT

OBJETIVOS

- Descrever a interrupção como uma chamada de sub-rotina forçada pelo hardware, em termos do ciclo de busca-execução do encontro 2.
- Identificar as variáveis compartilhadas entre o tratamento e o programa principal, e proteger as que precisam de `volatile` ou de atomicidade.
- Estimar latência e flutuação de latência, e decidir quais tarefas toleram cada uma.
- Explicar o repique de contato como fenômeno mecânico, e distinguir as soluções de hardware das de software.
- Implementar confirmação por tempo dentro do tratamento de um temporizador, sem bloquear o programa principal.

1 Cinco dívidas com a mesma forma

Encontro	O problema	Custo
3	A tela bloqueia enquanto o lá de 440 Hz precisa de uma inversão a cada 1,14 ms	1,3 ms
4	O programa espera o conversor sem ter o que fazer	60 ciclos
5	O recarregamento do Timer0 atrasa o intervalo seguinte	acumulativo
5	Estouros são perdidos se o laço demorar mais que o intervalo	silencioso
5	O CCP não alcança 440 Hz; o software alcança e bloqueia	—

Cinco problemas, uma forma só: **o processador precisa aparecer num instante determinado, e a única maneira que ele tinha era ir olhar.**

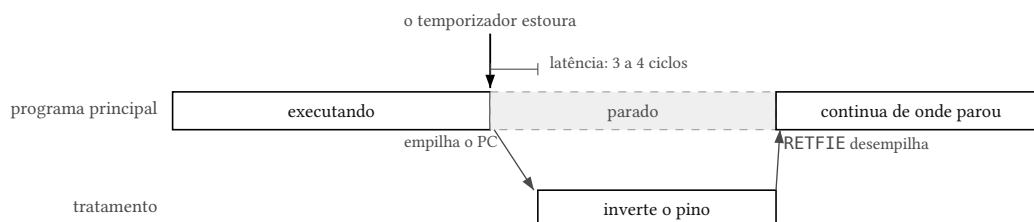
A Q4 da integradora pediu a descrição de um mecanismo — desvio assíncrono, independente do que estivesse rodando, com retorno ao ponto de origem. Ele existe, está no silício desde sempre, e este encontro é sobre ele.

2 O que é uma interrupção

Uma interrupção é uma **chamada de sub-rotina que o hardware faz sem pedir licença**. Quando a condição ocorre, ao final da instrução corrente o processador empilha o contador de programa, carrega nele um endereço fixo e continua executando dali. A instrução `RETFIE` desempilha e a execução volta exatamente para onde estava.

CONCEITO

Não há mágica nenhuma nisso, e o encontro 2 já tinha todas as peças: o PC é um registrador, e carregar outro valor nele é o que qualquer desvio faz. A novidade é apenas **quem** decide carregar.



O programa principal não é consultado, não coopera e não fica sabendo. Do ponto de vista dele, nenhuma instrução foi pulada — só levou mais tempo.

Figura 1: O programa principal não é consultado. Do ponto de vista dele, nenhuma instrução foi pulada — o programa apenas levou mais tempo para chegar ao fim.

O PIC18 tem dois endereços de desvio: 0x0008 para interrupções de alta prioridade e 0x0018 para as de baixa. São endereços fixos em silício, como o 0x0000 do reset.

E aqui volta o princípio do encontro 0 pela quarta vez. O bootloader ocupa o começo da Flash, **incluindo os endereços 0x0008 e 0x0018**. Quem responde à interrupção primeiro é ele, não a sua aplicação.

NESTE KIT

O bootloader precisa então reencaminhar o desvio para um endereço mais alto, onde o seu tratamento realmente está, e o arquivo de ligação da aplicação precisa concordar com esse endereço. É a mesma classe de erro do simulador que começava a executar no menor endereço do HEX: quando não bate, nada acusa — o tratamento simplesmente nunca roda.

Verificar antes do R7: qual deslocamento de vetor o bootloader do XM118 usa, e se o projeto de exemplo do curso já traz o arquivo de ligação correspondente.

2.1 Latência

Entre a condição ocorrer e a primeira instrução do tratamento executar passam-se alguns ciclos: terminar a instrução corrente, empilhar, carregar o PC. São três a quatro ciclos — menos de um microssegundo a 16 MHz.

O número que costuma importar não é a latência: é a **variação** dela.

ATENÇÃO

A latência mínima é fixa. A máxima depende do que estava rodando: se uma instrução de dois ciclos acabou de começar, espera-se mais um; se outro tratamento estiver em curso, espera-se ele terminar.

Para inverter um pino a 440 Hz, uma flutuação de um microssegundo sobre 1136 µs é irrelevante — 0,09%, ou menos de dois centésimos de semitom. Para medir a largura de um pulso, a mesma flutuação é o erro da medida.

3 O que o tratamento não pode estragar

O tratamento roda no meio do programa principal, sem que ele saiba. Duas coisas que funcionavam param de funcionar.

3.1 *volatile*

O compilador não sabe que o tratamento existe. Para ele, uma variável que o laço lê e nunca escreve é uma constante — e otimizar a leitura para fora do laço é correto.

CONCEITO

```
uint8_t pronto = 0;           /* escrito pelo tratamento */

while (!pronto) { }          /* laço infinito: o compilador
                              leu 'pronto' uma vez so   */
```

volatile é a declaração de que o valor pode mudar por fora do fluxo visível, e obriga a releitura a cada acesso. Sem ela o programa funciona em depuração, com otimização desligada, e trava com otimização ligada.

Este é um sintoma de bancada de primeira grandeza: **funciona no modo de depuração e trava no modo normal**. Quando isso acontecer, a primeira suspeita é uma variável compartilhada sem **volatile**.

NOTA

3.2 Atomicidade

volatile resolve a visibilidade, não a **indivisibilidade**.

CONCEITO

Um contador de dezesseis bits é lido em duas instruções, um byte de cada vez. Se a interrupção cair entre as duas, os dois bytes vêm de valores diferentes — lê-se um número que nunca existiu.

Com o contador em 0x00FF passando a 0x0100, uma leitura infeliz devolve 0x01FF: mais alto que os dois. Num relógio de milissegundos, isso é um salto de 256 ms para o futuro, uma vez a cada 256 incrementos.

```
volatile uint16_t ms = 0;

uint16_t ms_ler(void)
{
    uint16_t a, b;
    do {
        a = ms;
        b = ms;
    } while (a != b);    /* le duas vezes; aceita quando concordam */
    return a;
}
```

A alternativa é desabilitar a interrupção durante a leitura, o que é mais direto e tem um preço: a interrupção fica atrasada por esses poucos ciclos. Ler duas vezes até concordar não atrasa ninguém e custa uma releitura rara.

NOTA

Qual das duas usar depende de qual recurso está mais apertado — tempo de resposta ou ciclos. Essa pergunta reaparece o resto do semestre.

4 O ruído de contato

Uma chave mecânica não fecha uma vez: as lâminas se chocam, ricocheteiam e estabilizam. Durante alguns milissegundos o pino oscila entre os dois níveis.

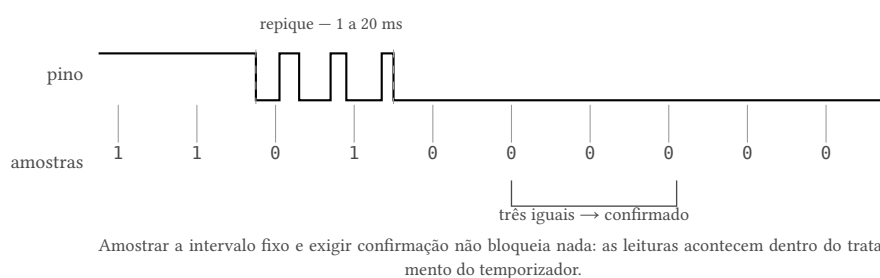


Figura 2: O repique é físico, dura de 1 a 20 ms e é característico **daquela** chave. A duração é medida, não copiada da internet.

Ligar um botão a uma entrada de interrupção sem tratamento é a receita para o defeito mais confuso da bancada: **um toque gera vinte eventos**.

ATENÇÃO

O setpoint sobe cinco graus de uma vez, o contador pula, o menu atravessa três telas. E o número de eventos muda a cada toque, porque o repique é aleatório — o que torna o defeito quase impossível de reproduzir de propósito.

No PICSIMLab o botão é ideal: fecha uma vez e pronto. O simulador **mente** aqui, e saber onde ele mente é parte do conteúdo.

DIVERGÊNCIA

Repique se estuda na bancada, com osciloscópio. O **algoritmo** que o filtra, esse sim, se estuda no simulador — injetando uma sequência de amostras e verificando o que o filtro decide. Cada ferramenta para a metade que ela faz bem.

4.1 Duas famílias de solução

	Hardware	Software
Como	RC na entrada mais disparador Schmitt	Confirmação por tempo
Custo	Dois componentes por botão	Algumas linhas, uma vez

Escala	Ruim: dezesseis botões, trinta e dois componentes	Boa: o mesmo código serve para todos
Quando escolher	Quando o sinal precisa estar limpo antes do pino — entrada de interrupção, de captura ou de contagem	Quando o pino é lido por amostragem

O que não fazer: detectar a borda e chamar `__delay_ms(20)`.

CONCEITO

Funciona, e desfaz tudo que este encontro construiu — bloqueia o processador por vinte milissegundos, que é quinze vezes o pior bloqueio que o curso tinha até agora. Trocar espera cega por espera cega maior não é solução.

O que fazer: amostrar o pino a intervalo fixo, dentro do tratamento do temporizador que já existe, e só aceitar a mudança depois de N leituras consecutivas iguais. Com amostragem a 5 ms e $N = 3$, a confirmação leva 15 ms e custa algumas instruções por amostra.

5 O código

```
#define PRECARGA_5MS 45536u /* 65536 - 20000: 5 ms a 250 ns */

volatile uint16_t ms5 = 0; /* tempo, em passos de 5 ms */
volatile uint8_t botao = 1; /* estado confirmado (1 = solto) */
volatile uint8_t borda = 0; /* pulso de "acabou de ser pressionado" */

void __interrupt(high_priority) tratar(void)
{
    static uint8_t candidato = 1;
    static uint8_t iguais = 0;

    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;
        TMR0H = (uint8_t)(PRECARGA_5MS >> 8);
        TMR0L = (uint8_t)(PRECARGA_5MS & 0xFF);

        ms5++;

        uint8_t agora = PORTBbits.RB0; /* ADCON1 = 0x0F, encontro 4 */

        if (agora != candidato) {
            candidato = agora;
            iguais = 1;
        } else if (iguais < 3) {
            iguais++;
            if (iguais == 3 && candidato != botao) {
                botao = candidato;
                if (botao == 0) {
                    borda = 1; /* pressionado, confirmado */
                }
            }
        }
    }
}
```

Três detalhes que valem a leitura atenta.

NOTA

O tratamento **não** espera, não escreve no display e não converte nada. Ele lê um pino, mexe em três variáveis e sai. Tudo que é lento continua no programa principal.

`candidato` e `iguais` são `static` e não são `volatile`: só o tratamento os enxerga. `volatile` é para o que atravessa a fronteira, e marcar tudo como `volatile` desperdiça otimização sem comprar segurança.

`borda` é um pulso que o programa principal consome — lê e zera. É a forma mais simples de um sinal de mão única, e a que menos exige do laço principal.

A melodia, agora. Com o tratamento invertendo o pino do buzzer, o lá de 440 Hz precisa de meio período de 1136,4 μ s, ou 4545 ciclos. Pré-carga: $65536 - 4545 = 60991$.

CONCEITO

O erro fica em 4545 contra 4545,45 ciclos — 0,01%, ou cerca de um terço de centésimo de semitom, inaudível.

E, o que importa mais: a nota continua correta **durante** a atualização de tela. O bloqueio de 1,3 ms do display não impede a interrupção de acontecer; ele apenas adia o programa principal, que não é quem gera a nota.

6 Previsão para o R7

PREVISÃO

P1. Você vai ligar o osciloscópio no botão e capturar um toque. Quantos milissegundos de repique espera ver? E quantas transições?

P2. Antes de filtrar, você vai contar eventos por toque num contador simples. Escreva quantos espera. Depois toque dez vezes: o número se repete?

P3. Com amostragem a 5 ms e confirmação de 3, qual é o atraso entre encostar no botão e o programa reagir? Esse atraso é perceptível?

P4. Se você aumentar a confirmação para 10 amostras, o que melhora e o que piora? Dê os dois números.

P5. Você vai medir o repique de dois botões diferentes da placa. Espera o mesmo número nos dois? Justifique antes de medir.

Leve esta folha preenchida e a captura de osciloscópio do P1 — ela vira a entrada do simulador na segunda metade do roteiro.

SEM NOTA

7 Exercícios

Exercício 7.1. Um programa usa `uint16_t` ms incrementado no tratamento e lido no laço principal para medir intervalos. Ele funciona, mas de vez em quando registra um intervalo absurdo, sempre positivo e sempre grande.

TAREFA

(a) Qual é o defeito?

(b) Por que o erro é sempre para mais, e nunca para menos?

(c) Corrija sem desabilitar interrupções.

Exercício 7.2. Um colega resolve o repique assim: no tratamento da interrupção por mudança de estado em `PORTB`, chama `__delay_ms(20)` e depois lê o pino de novo.

TAREFA

(a) Funciona?

(b) Que dano isso causa ao resto do sistema?

(c) Qual é a diferença conceitual entre essa solução e a da aula, além do tempo?

Exercício 7.3. Você precisa medir a largura de um pulso externo com resolução de 1 μ s, e considera fazê-lo por interrupção: anotar o tempo na borda de subida e na de descida.

TAREFA

- (a) A latência atrapalha? Justifique.
- (b) E a variação da latência?
- (c) Que recurso do chip resolveria isso melhor, e por quê?

Exercício 7.4. O tratamento roda a cada 5 ms e consome 40 ciclos. O programa principal atualiza a tela, o que custa 5 200 ciclos de bloqueio.

TAREFA

- (a) Que fração do processador o tratamento consome?
- (b) Durante a atualização de tela, quantas vezes o tratamento roda?
- (c) O programa principal percebe alguma coisa?

No encontro 8: estágio de potência, atuadores e fontes de reset. Até aqui os pinos moveram LEDs e um display, que consomem miliampêres. O aquecedor e o relé consomem ampêres, e nenhum pino do PIC18F4550 os aciona diretamente.

NOTA

E vem junto a pergunta que a bancada faz sozinha assim que o primeiro atuador de verdade comuta: **por que ele reiniciou?**